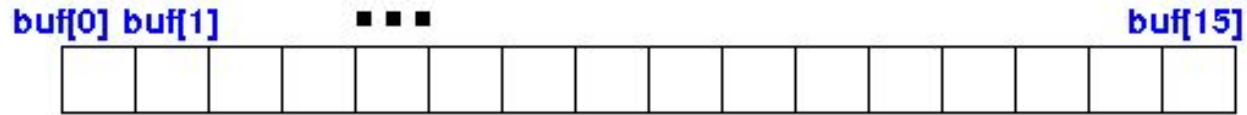
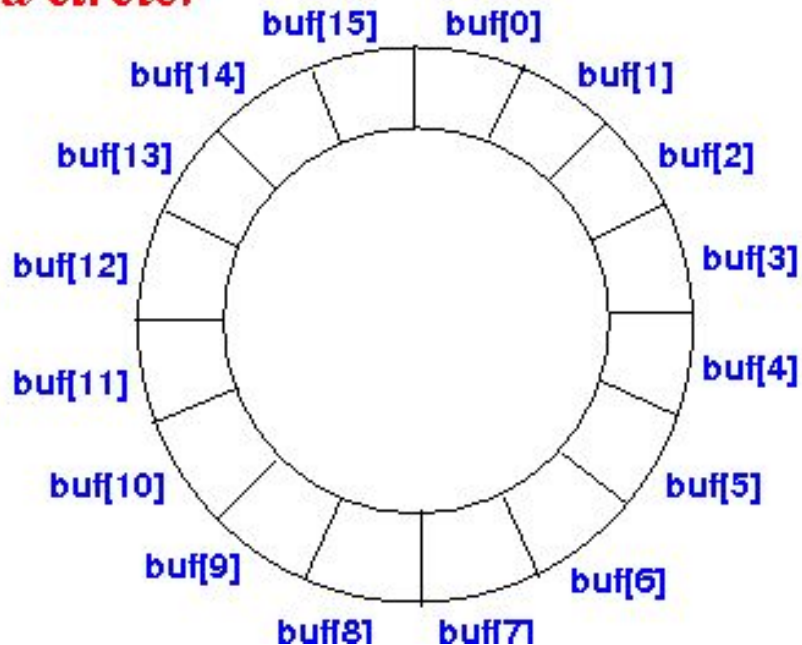


Array queue

Array:



Pretend array is a circle:



- **Circular array** = a data structure that used a **array** as if it were **connected end-to-end**
- Schematically:
- This **data structure** is also known as:

- **Circular buffer**
- **Cyclic buffer**
- **Ring buffer**

A **circular buffer** has **2 "pointers"** (or indices):

- **Read pointer** = the **index** of the **circular array** used by a **read operation**

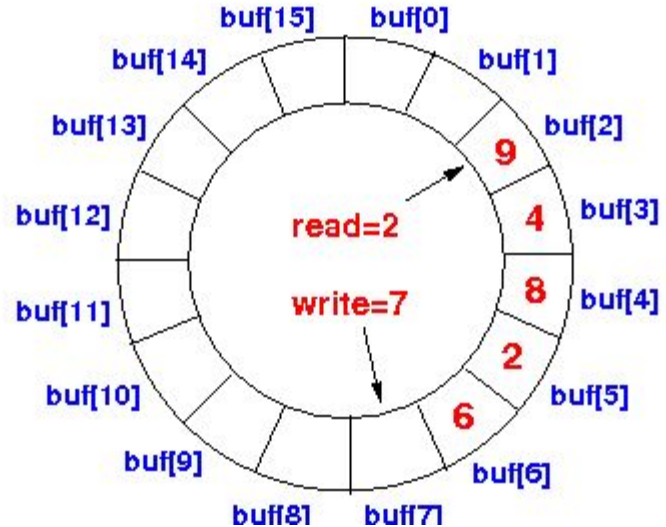
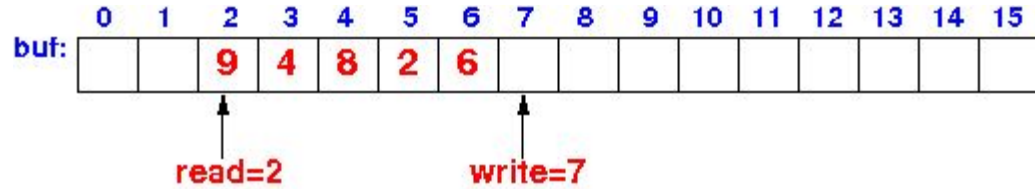
Example:

- In the **figure**, a **read operation** will return the **value 9** in **buf[2]**
(because the **read pointer read = 2**)

- **Write pointer** = the **index** of the **circular array** used by a **write operation**

Example:

- In the **figure**, a **write operation** will update the **array element buf[7]**
(because the **write pointer write = 7**)



- The **read** and **write** operations will **advance** their **corresponding pointers**
(Because you don't want the **read** and **write** operations to read/write the **same array element** over and over again....)

- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, **0**, 1, 2, 3, and so on...

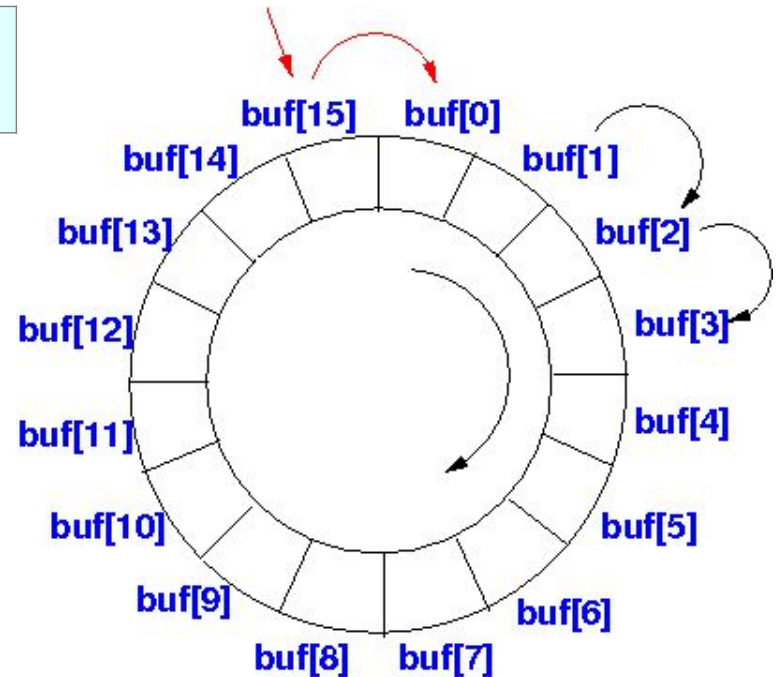
pointers will **advance** in the **following manner**:

because the **indices** will **wrap around**:

It is **very easy** to increase an **index** in a **wrap around manner**:

$$\text{index} = (\text{index} + 1) \% N$$

where **N** = the number of indices



Buffer is assumed to be full !!!

We *assume* that the **circular buffer** is *full* when:

- There is *one empty slot* left in the circular buffer:

